DATA ENGINEERING MASTERCLASS

Retail Sales Data Engineering Pipeline

From Raw Kaggle Transactions to Analytics-Ready PostgreSQL Warehouse, Parquet Data Lake, and Azure Cloud Storage

---

Author / Candidate: Vanshika Patel
Target Role: Associate / Junior Data Engineer
Core Technologies: Python 3.14, PostgreSQL 17, Apache Arrow / Parquet, SQLAlchemy 2, Psycopg 3, Pytest, Azure ADLS Gen2
Document Purpose: Exhaustive tutor walkthrough explaining core data engineering principles, technical decisions, complete source code architecture, and interview defense strategies.

HOW TO USE THIS GUIDE

This document is divided into 5 structured sections. Read **Part I** to master the fundamental concepts as if taught by a senior tutor. Study **Part II** to understand the exact business problem and dataset. Review **Part III** for an exhaustive code-by-code walkthrough. Read **Part IV** to learn why specific architecture decisions were made. Finally, use **Part V** to rehearse your spoken responses for job interviews.

PART I: Foundational Concepts & The Tutor's 'Why'

1.1 What is Data Engineering?

To understand data engineering, consider a water treatment plant. Raw water from a river is muddy, contaminated, and flows unpredictably. The municipal facility builds intake pumps, filtration beds, chemical treatments, and storage reservoirs before piping clean, safe drinking water to houses and factories. **In the tech industry, raw data is like muddy river water.** It arrives in different formats (Excel, CSV, JSON, streaming APIs) with corrupt characters, missing values, duplicates, and cancelled transactions. Data Engineers build the automated software pipelines, storage tanks (Data Lakes/Warehouses), and quality filtration systems that turn this raw sludge into trusted, analytics-ready data for Business Analysts, Data Scientists, and executive dashboards.

1.2 The Core Paradigm: ETL vs. ELT (And Why We Use a Hybrid)

One of the most frequent interview questions asks you to contrast ETL and ELT:

- **ETL (Extract, Transform, Load):** Raw data is extracted from the source, transformed (cleaned, filtered, enriched) in an intermediate compute engine (like Python/Pandas or Spark), and only the clean data is loaded into the destination database. *Advantage:* Keeps junk data out of the warehouse. *Disadvantage:* Transformations are limited by Python memory.
- **ELT (Extract, Load, Transform):** Raw data is loaded directly into a high-capacity database/warehouse first, and SQL queries are used to transform it inside the database. *Advantage:* Extremely fast for massive datasets; raw data is preserved. *Disadvantage:* Staging tables must accept loose types (like everything stored as TEXT).

WHY OUR PROJECT USES A HYBRID ETL + ELT ARCHITECTURE

In our pipeline, we do not choose one dogmatically. We use **ETL for data quality** and **ELT for business analytics**:
1. **The ETL Stage (Python):** We extract Excel records and run 8 deterministic validation rules in Python. We separate clean from rejected rows *before* touching the database. This guarantees that corrupt values (e.g., text prices) never reach PostgreSQL.
2. **The ELT Stage (PostgreSQL SQL):** Once clean data sits in PostgreSQL staging, we use modular SQL views (analytics.fact_sales) to perform business classifications (sales vs. returns vs. cancellations, date truncations, aggregates). This leverages PostgreSQL's relational engine and indexing for fast set-based querying.

1.3 Storage Formats: CSV vs. Parquet (Why Columnar Storage Matters)

Why didn't we just save our processed data as a clean CSV file? Because CSV is a **row-oriented plain-text format**, while modern data analytics demands **columnar binary storage** like Apache Parquet.

Feature	Row-Oriented (CSV)	Columnar Binary (Apache Parquet)
Layout on Disk	Stores complete rows sequentially: row1, row2, row3.	Stores each column sequentially: all quantities together, all prices togeth
Analytical Query	To sum 1 column, the disk must read every single byte of every row.	Projection Pushdown: Only reads the single requested column, s
Compression	Very low compression because adjacent values have mixed types.	High Compression (Snappy): Adjacent values are identical types
Schema & Types	No schema metadata. Everything is text; types must be re-inferred.	Strict binary metadata embedded in the file footer (int64, float64, timestar
Industry Role	Good for human-readable audit logs (like rejected records).	The standard storage format for Data Lakes, Snowflake, Databricks, and

1.4 Relational Database vs. Data Lake: Why We Have Both

A common point of confusion is: *'If we have Parquet files, why do we need PostgreSQL? And if we have PostgreSQL, why do we need Parquet?'*
In modern enterprise data platforms, this is known as a **Lakehouse Pattern**:
• **The Data Lake (Parquet on Azure ADLS Gen2):** Acts as our durable, cheap, scalable object storage layer. It can hold historical snapshots, massive partitions, and can be queried directly by distributed compute engines (Spark, Synapse, Databricks) without paying database license fees.
• **The Relational Warehouse (PostgreSQL):** Serves business intelligence analysts who need immediate sub-second SQL queries, ACID transaction guarantees, role-based access control, and foreign key relations for business dashboards.

1.5 Database Drivers & Abstraction: SQLAlchemy vs. Psycopg 3

Connecting Python to PostgreSQL requires two cooperating layers:
1. **Psycopg 3 (The Driver):** This is the low-level library that speaks PostgreSQL's binary wire protocol over TCP network sockets. It handles socket connections, SSL handshakes, and query transmission.
2. **SQLAlchemy (The Abstraction Layer):** A high-level SQL toolkit. You never want individual functions opening raw database connections because creating TCP connections is expensive. SQLAlchemy manages a **Connection Pool** (reusing pre-opened connections), abstracts dialect differences, handles automatic rollbacks on error, and builds safe connection URLs via quote_plus.

PART II: Business Problem & Data Journey

2.1 The Kaggle Online Retail II Dataset

The project uses the real-world **Online Retail II** dataset from the UCI Machine Learning Repository. It contains **525,461 transaction line items** from an e-commerce gift retailer based in the United Kingdom, spanning two years (2009 to 2011).

Column Name	Raw Example	Data Type	Business Role & Meaning
Invoice / InvoiceNo	536365, C536379	VARCHAR	Order ID. Invoices starting with 'C' indicate an order cancellation.
StockCode	85123A, POST	VARCHAR	Product SKU code. Includes service codes like POST (postage).
Description	WHITE HANGING HEART...TEXT		Product name. Missing in ~2-3% of rows.
Quantity	6, -1, 24	BIGINT	Units sold. Negative values represent customer returns/credit notes.
InvoiceDate	2010-12-01 08:26:00	TIMESTAMP	Exact transaction timestamp.
Price / UnitPrice	2.55, 0.0, -10.0	FLOAT	Unit price in GBP (£). Zero price is valid (free sample); negative is invalid.
Customer ID	17850.0, NaN	FLOAT / NULL	Customer identifier. Missing in ~25% of rows due to guest checkout.
Country	United Kingdom, France	VARCHAR	Country of the purchasing customer.

2.2 Critical Business Rules & Logic Handled

Many junior developers make naive mistakes with this dataset. Our pipeline implements strict, documented business rules:

1. **Returns vs. Cancellations:** Invoices starting with 'C' are cancellations. Negative quantities on regular invoices are customer returns. Both are kept in the clean dataset! Why? Because returns and cancellations are real business events that reduce net revenue. If you reject them, you falsely inflate company profits.
2. **Guest Checkouts (Missing Customer IDs):** Over 25% of transactions have no Customer ID because customers checked out as guests. If you run df.dropna(), you eliminate 130,000+ sales and £2M+ in revenue. We preserve missing Customer IDs as database NULL values.
3. **Zero Price vs. Negative Price:** An item with price = 0 is valid (a free promotional sample). An item with price < 0 is corrupt accounting data and is routed to the rejected dataset.
4. **Exact Duplicate Invoices:** A composite key of (invoice, stockcode, invoicedate, quantity, price) identifies accidental duplicate line entries. The first line is kept; duplicate copies are logged to the rejected CSV with reason duplicate;.
5. **Derived Line Amount:** Computed as: line_amount = quantity * price. Returns yield negative line amounts, which naturally subtract from gross sales when summed.

THE 100% RECONCILIATION INVARIANT

A senior data engineering principle enforced in our pipeline is the mathematical reconciliation invariant:
Total Source Input Rows = Clean Output Rows + Rejected Output Rows
If 1,010 rows enter the pipeline, and 1,000 pass while 10 are rejected, the pipeline verifies 1,010 == 1,000 + 10. If a single row is dropped silently, the pipeline logs an integrity warning. This ensures total auditability for compliance.

PART III: Step-by-Step Code Walkthrough

3.1 Module 1: Centralized Configuration (src/config.py)

Why it exists: Hardcoding file paths or database passwords inside pipeline scripts causes security leaks and environment fragility. config.py is the single source of truth for the entire application.

Key Code Structure & Decisions:

• PROJECT_ROOT = Path(__file__).resolve().parent.parent dynamically detects the root directory, so the script works no matter which terminal folder you run it from.

• urllib.parse.quote_plus is wrapped around the database password. This prevents special characters like @ or # from corrupting the SQLAlchemy connection URL.

• __repr__ masks passwords: password='***', so printing the config never leaks credentials in logs.

```
# src/config.py snippet
def get_connection_url(self, db_name: str | None = None) -> str:
target_db = db_name or self.name
# quote_plus safely encodes passwords with special characters (@, #, %)
pw_encoded = quote_plus(self._password)
return f"postgresql+psycopg://{self.user}:{pw_encoded}@{self.host}:{self.port}/{target_db}"

def __repr__(self) -> str:
# Zero-secret leakage: password masked in logs
return f"DatabaseConfig(host={self.host!r}, port={self.port}, db={self.name!r}, user={self.user!r},
password='***')"
```

3.2 Module 2: Ingestion & Extraction (src/etl.py)

Why it exists: The raw Kaggle file contains multiple sheets (Year 2009-2010 and Year 2010-2011). Reading 500K rows twice or loading the wrong sheet breaks the pipeline.

Key Code Structure & Decisions:

• inspect_workbook(path) uses openpyxl.load_workbook(..., read_only=True) to read only sheet metadata in milliseconds without parsing any rows.

• _read_sheet() uses dtype=object to prevent Pandas from coercing numbers prematurely.

• Adds df['_sheet_source'] = sheet_name to every sheet before concatenation to maintain full data lineage.

```
# src/etl.py snippet
def extract(source_path: Path) -> ExtractResult:
sheet_names = inspect_workbook(source_path) # openpyxl read_only inspection
raw_frames = []
for sheet in sheet_names:
df_raw = _read_sheet(source_path, sheet)
df_norm = normalize_headers(df_raw)
df_norm["_sheet_source"] = sheet # Data Lineage Tracking
raw_frames.append(df_norm)
combined = pd.concat(raw_frames, ignore_index=True, sort=False)
return ExtractResult(df=combined, source_row_count=len(combined), sheets_loaded=sheet_names)
```

3.3 Module 3: Validation & Quality Gate (src/validation.py)

Why it exists: This is the heart of the ETL layer. It separates clean from rejected records using 8 business validation rules.

Key Code Structure & Decisions:

• flag_missing_required() flags records where invoice, stockcode, quantity, price, or invoicedate are null or blank.

• flag_non_numeric() attempts numeric coercion with errors='coerce'. Any value that was present but becomes NaN is tagged as non_numeric_price;.

• flag_duplicates() evaluates (invoice, stockcode, invoicedate, quantity, price) to identify duplicate transactions.

• Automatically tags cancellations: df['is_cancellation'] = df['invoice'].str.startswith('C').

• Calculates: df['line_amount'] = df['quantity'] * df['price'].

```
# src/validation.py snippet
# Composite key deduplication
dup_mask = df.duplicated(subset=['invoice', 'stockcode', 'invoicedate', 'quantity', 'price'],
keep="first")
dup_df = df[dup_mask].copy()
dup_df['rejection_reason'] = "duplicate; "

# Derived cancellation flag and financial line amount
passing['is_cancellation'] = passing['invoice'].astype(str).str.strip().str.upper().str.startswith("C")
passing['line_amount'] = passing['quantity'] * passing['price']
```

3.4 Module 4: Parquet Lakehouse Storage (src/transform.py)

Why it exists: Converts the clean validated DataFrame into an optimized, binary, typed Apache Parquet file for downstream analytical processing.

Key Code Structure & Decisions:
• Explicit PyArrow Schema: Defines exact data types (string, int64, timestamp[us], float64) to prevent schema drift.
• pq.write_table(..., compression='snappy', row_group_size=100_000) writes columnar row groups of 100K rows for optimal disk I/O.
• write_rejected_csv() uses encoding='utf-8-sig'. The Byte Order Mark (BOM) ensures that business users opening the rejected file in Microsoft Excel on Windows do not see garbled characters.
• verify_parquet_roundtrip() immediately reads the Parquet file back and verifies row counts and column parity.

```
# src/transform.py snippet
table = pa.Table.from_pandas(df, preserve_index=False)
pq.write_table(
table, str(output_path),
compression="snappy", # Fast compression with ~80% size reduction
row_group_size=100_000, # 100K-row groups for efficient projection scans
write_statistics=True,
)
# Automated roundtrip verification
assert verify_parquet_roundtrip(df, output_path) == True
```

3.5 Module 5: PostgreSQL Loading (src/load_postgres.py)

Why it exists: Loads the clean Parquet records into relational PostgreSQL staging tables while managing connections safely.

Key Code Structure & Decisions:
• create_database_if_missing() connects to the default postgres administrative database with AUTOCOMMIT isolation to check if retail_dw exists. If missing, it creates the database automatically.
• load_staging() uses df.to_sql(..., method='multi', chunksize=5000). Instead of inserting 500K individual rows or one massive query that crashes RAM, it batches records into 5,000-row chunks.
• Uses if_exists='replace' to provide an **idempotent full-refresh staging strategy**.

```
# src/load_postgres.py snippet
df.to_sql(
name="stg_transactions",
schema="staging",
con=engine,
if_exists="replace", # Idempotent: dropping and reloading guarantees exact state
index=False,
chunksize=5000, # 5K-row chunks to prevent database timeouts & OOM
method="multi", # Batches multi-row INSERT statements
)
```

3.6 Module 6: SQL Analytics & Views (sql/)

Why it exists: The database layer performs the ELT business transformations using modular SQL views.

- **sql/01_create_schema.sql:** Creates separate staging (ETL landing) and analytics (business layer) schemas.
- **sql/02_create_staging.sql:** Adds B-tree performance indexes on invoicedate, customerid, country, and is_cancellation.

- **sql/03_transform_sales.sql:** Builds analytics.fact_sales, classifying transaction types ('sale', 'return', 'cancellation', 'free_item') and extracting year, month, and day-of-week.
- **sql/04_analytics_views.sql:** Builds 5 business KPI views: Daily Net Sales, Monthly Trends (AOV), Top Products by Revenue, Country Sales, and Return Analysis.
- **sql/05_quality_checks.sql:** Runs automated post-load SQL assertions verifying row counts, checking for null values, and proving line_amount = quantity * price.

3.7 Module 7: Orchestrator & CLI (src/run_pipeline.py)

Why it exists: Coordinates all pipeline stages in sequence, parses CLI arguments (--local-only, --create-db, --upload-azure), logs execution, and writes immutable audit records to logs/audit_runs.jsonl.

```
# Run the pipeline from terminal:
python -m src.run_pipeline --local-only

# Sample audit record written to logs/audit_runs.jsonl:
{
"run_id": "3588b8ce",
"start_time": "2026-10-25T14:25:09Z",
"end_time": "2026-10-25T14:25:11Z",
"source_row_count": 1010,
"clean_row_count": 1000,
"rejected_row_count": 10,
"duplicate_count": 2,
"status": "SUCCESS",
"local_only": true
}
```

3.8 Module 8: Optional Cloud Lakehouse Upload (src/azure_upload.py)

Why it exists: Uploads processed Parquet and rejected CSV files to Microsoft Azure Data Lake Storage Gen2 (ADLS Gen2). Uses dynamic imports so local development works 100% offline without failing if Azure packages or credentials are missing.

3.9 Automated Testing Suite (tests/)

Our test suite contains **89 automated tests** across 5 modules that execute in under 3 seconds using pytest. We test using small synthetic DataFrames (mock data) so developers can verify code changes without downloading 500K rows or running PostgreSQL:

- test_validation.py (40+ tests): Asserts header mapping, missing invoice rejection, negative price rejection, return preservation, and reconciliation invariant.
- test_transform.py: Verifies Parquet read/write roundtrip, idempotency of overwrites, and CSV writing.
- test_config.py: Tests URL encoding, missing password errors, and password masking.
- test_azure.py: Tests graceful skipping when Azure credentials are not present.
- test_etl.py: Tests multi-sheet discovery, missing file errors, and schema compatibility.

PART IV: Architectural Trade-Offs & Decisions

Architectural Decision	What We Chose	What We Rejected & Why
Processing Engine	Python 3 + Pandas + PyArrow	Rejected Apache Spark / PySpark: The dataset is ~525K rows (~100MB). S
Pipeline Strategy	Idempotent Full Refresh	Rejected Complex Streaming / CDC: Retail transaction logs arrive in daily/
Data Lake File Format	Apache Parquet (Snappy)	Rejected Raw CSV / JSON: CSV has no type safety and takes 80MB. Parq
Database Staging Load	SQLAlchemy Multi-Insert (Chunksize=5000)	Rejected Single-row INSERT / Massive Bulk Load: Single-row inserts are t
SQL Analytics Layer	Modular SQL Database Views	Rejected Python Aggregations: Computing AOV and monthly trends in SQ
Error Handling Strategy	Rejection Routing (Input = Clean + Rejected)	Rejected Silent df.dropna(): Dropping bad rows silently destroys data lineag

4.2 Failure Modes & How the Pipeline Recovers

- **PostgreSQL Offline / Refused:** The pipeline has a --local-only flag. If PostgreSQL is stopped, it processes data, writes Parquet and rejected CSV files, and computes analytics from Parquet without crashing.
- **Corrupt / Password Failure:** config.py validates environment variables at startup and catches auth errors with friendly instructions pointing to troubleshooting guides.
- **Schema Drift (New Column Names):** normalize_headers() handles case differences (InvoiceNo, invoiceno, Invoice), strips spaces, and aliases common column names.
- **Special Characters in Password:** quote_plus() ensures passwords containing @, #, %, or / do not corrupt the SQLAlchemy connection URL.

PART V: Interview Masterclass & Spoken Defense

5.1 Spoken Elevator Pitches (Practice Out Loud)

THE 30-SECOND PITCH (FOR SCREENING CALLS)

'I built an end-to-end retail data engineering pipeline in Python and PostgreSQL that processes over 525,000 real-world e-commerce transactions. The pipeline extracts multi-sheet Excel data, enforces 8 automated data quality checks, and stores clean data in a Snappy-compressed Parquet lakehouse format, saving 80% storage space. It then batch-loads clean records into an indexed PostgreSQL staging warehouse, where modular SQL views calculate daily sales, return rates, and product rankings. The entire pipeline is idempotent, covered by 89 automated pytest tests, and includes optional Azure ADLS Gen2 cloud synchronization.'

THE 90-SECOND TECHNICAL PITCH (FOR TECHNICAL ROUNDS)

'In e-commerce datasets, raw transaction logs are messy—they contain duplicate orders, cancelled invoices, negative quantities, and missing customer IDs from guest checkouts. If loaded blindly, financial revenue metrics get distorted.
To solve this, I designed a hybrid ETL/ELT architecture:
1. In the ETL layer, Python and Pandas inspect the Excel workbook metadata using openpyxl, normalize column headers, and enforce 8 business validation rules. We separate clean from invalid records, routing rejected records to an audited CSV with explicit error tags to guarantee our reconciliation invariant: Input = Clean + Rejected.
2. For the Data Lake layer, clean records are cast to explicit PyArrow types and written to columnar Parquet with Snappy compression, enabling projection pushdown.
3. For the Warehouse layer, data is chunk-loaded into PostgreSQL staging using SQLAlchemy and Psycopg 3. Downstream SQL views classify records into sales, returns, and cancellations, computing key metrics like Net Sales and Average Order Value.
The project is production-hardened with zero-secret logging, automated audit tracking in JSONL, and 89 automated unit tests running in under 3 seconds.'

5.2 Top 5 Tough Interview Questions & How to Answer

Q1: Why did you keep negative quantities in the clean dataset?

Because in retail accounting, negative quantities represent legitimate customer returns or credit notes. If an invoice starts with 'C', it is an order cancellation; if a normal invoice has a negative quantity, it is a return. If you automatically reject negative quantities, you overstate company sales. In my pipeline, they are preserved and flagged with `is_cancellation` so SQL views subtract returns from gross sales to calculate true Net Revenue.

Q2: How did you handle the 25% missing Customer IDs?

About 25% of rows had no Customer ID because online customers frequently check out as guests. Dropping these rows with `dropna()` would destroy 25% of sales volume. I classified `customerid` as an optional field, storing it as database `NULL`. When calculating customer-specific metrics like Customer Lifetime Value (CLV), queries filter `WHERE customerid IS NOT NULL`, while total revenue queries include all transactions.

Q3: What does idempotency mean in your pipeline?

Idempotency means running the pipeline multiple times on the same input yields the exact same state without duplicating records. Parquet files are written with atomic overwrite semantics. In PostgreSQL staging, we use a transactional full-refresh strategy (`if_exists='replace'` in chunked batches). Downstream SQL transformations are implemented as database VIEWs (`CREATE OR REPLACE VIEW`), so they always compute dynamically over staging data.

Q4: Why Parquet instead of CSV for clean data?

Parquet is a columnar binary format with Snappy compression. A 500K-row dataset takes ~18MB in Parquet versus ~85MB in CSV (~80% reduction). More importantly, Parquet enables projection pushdown: analytical queries like `SELECT SUM(line_amount) FROM sales` only read that single column from disk, skipping all other columns. It is also the native standard for cloud tools like Databricks and Azure Synapse.

Q5: Why did you write 89 automated tests for a small project?

Automated testing separates amateur scripts from professional software engineering. In real data engineering, pipelines process critical financial data. A bug in date parsing or duplicate handling can silently miscalculate revenue. My 89 unit tests use small synthetic DataFrames to verify edge cases (negative prices, missing invoices, cancellations, date formats, roundtrip I/O) in under 3 seconds without needing a live database.